

PROYECTO DE LABORATORIO NO. 1

Modalidad: Individual

Valor: 10 puntos

Lenguaje: C#

Descripción general:

El estudiante desarrollará una aplicación de consola en C# que resuelva un problema real de gestión de información, aplicando de forma integrada las estructuras de datos estudiadas durante la primera parte del curso: listas enlazadas, árboles BST, árboles AVL, árboles B, árboles B+ y montículos (min heap y max heap).

El objetivo no es solo que el programa funcione, sino que el estudiante justifique de manera técnica la elección de cada estructura de datos según las operaciones más frecuentes del problema (búsqueda, inserción, eliminación, acceso y recorrido de la estructura). Para este proyecto en particular, es obligatorio implementar y aplicar un Árbol B+ junto con un Min Heap y un Max Heap, según se detalla en los requisitos técnicos.

Temas propuestos:

Elija uno de los siguientes escenarios:

- **Catálogo de biblioteca:** registra los libros de una biblioteca, los cuales deben estar indexados por un código único para permitir búsquedas rápidas. Cada libro debe tener sus datos completos: código, título, autor, categoría, cantidad de copias disponibles y número de veces que ha sido prestado. Como puede existir más de una copia de cada libro, el sistema permite gestionar múltiples préstamos del mismo título, registrando cada préstamo y devolución para mantener actualizada la cantidad de copias disponibles. El programa permite registrar libros, buscar un libro por su código y generar un listado de todo el catálogo ordenado por título. Además, el sistema debe poder mostrar los libros más prestados de la biblioteca, para identificar cuáles son los más solicitados por los usuarios.
- **Tabla de posiciones de jugadores del mundial:** almacena los datos de los jugadores que participan en el Mundial junto con sus estadísticas de desempeño, como minutos jugados, goles, asistencias, tarjetas recibidas y partidos disputados. Cada jugador debe tener sus datos completos: nombre, selección a la que pertenece, posición en la que juega y sus estadísticas acumuladas. El programa permite registrar jugadores, actualizar las estadísticas de un jugador conforme avanza el torneo y buscar rápidamente las estadísticas de un jugador en específico. Además, el sistema debe poder generar el top 5 de jugadores en cada categoría (por ejemplo, los cinco máximos goleadores, los de más asistencias o los de más minutos jugados), así como mostrar el listado general de todos los jugadores ordenados por la categoría que se elija.
- **Control de candidatos:** guarda los datos de los candidatos a puestos de la empresa SISCOMERPROSA. En la empresa existen varios puestos, como mantenimiento, técnicos de reparación, operadores de maquinaria, desarrolladores, jefes técnicos, entre otros. Cada

candidato debe tener sus datos completos: nombre, lugar de origen, a qué puesto aplica y cuál es su pretensión salarial. El programa permite registrar candidatos, buscar un candidato específico y mostrar todos los candidatos de un puesto determinado, así como un registro general de todos los candidatos. Además, para cada puesto debe poder identificarse rápidamente al candidato con la menor pretensión salarial, o generar un listado de los candidatos ordenados por pretensión salarial.

Requisitos técnicos mínimos:

El proyecto debe cumplir con lo siguiente para ser válido:

1. **Estructuras obligatorias:** El programa debe implementar y utilizar correctamente un Árbol B+, un Min Heap y un Max Heap para resolver el escenario elegido. Cada estructura debe estar justificada según la operación que resuelve dentro del problema.
2. **Operaciones:** Incluir las operaciones básicas correspondientes a cada estructura: insertar, buscar, eliminar, imprimir y recorrer.
3. **Datos dinámicos:** Los datos no deben estar quemados en el código. Deben cargarse por medio de archivos .csv, .txt o ingresarse manualmente por medio de un menú.
4. **Menú interactivo:** El programa debe ofrecer un menú de consola claro que permita ejecutar todas las operaciones indicadas.
5. **Implementación propia:** La funcionalidad del programa debe ser implementada en su totalidad por el estudiante. No se permite utilizar clases o estructuras de colecciones nativas de .NET (por ejemplo, List<T> como estructura principal, Dictionary, SortedDictionary, SortedSet o PriorityQueue<TElement, TPriority>) para resolver la lógica del Árbol B+ o de los montículos. El uso de tipos nativos se limita a soporte auxiliar (lectura de archivos, manejo de cadenas), nunca a reemplazar la lógica de las estructuras solicitadas.

Entregables:

1. Código fuente en C#, debidamente comentado.
2. Manual técnico: incluye diagrama de flujo, diagrama de clases y explicación de la implementación, justificando la elección de cada estructura de datos utilizada. Se entrega en formato PDF.
3. Manual de usuario: instrucciones claras del funcionamiento del programa, incluyendo el uso de cada opción del menú.
4. Demostración funcional y defensa individual del proyecto ante el docente.

Consideraciones

- El proyecto se realizará de forma individual.
- La solución debe compilar correctamente para tener derecho a calificación y/o revisión funcional.
- La entrega se realizará únicamente en el espacio habilitado en el portal.
- Durante la revisión podrán solicitarse cambios de funcionalidad para comprobar el dominio.

La copia parcial o total tendrá una calificación de cero puntos y se notificará a la coordinación correspondiente.

Política de uso de IA y dominio individual del proyecto

El uso de herramientas de Inteligencia Artificial (IA) está permitido únicamente como guía de apoyo o consulta. No sustituye el proceso de análisis, diseño ni programación que debe realizar el estudiante.

Todo lo que se presente en el proyecto (análisis, diagrama de flujo, diagrama de clases, código y decisiones implementadas) debe poder ser explicado, justificado y defendido individualmente por el estudiante.

Durante la defensa presencial, cada estudiante deberá demostrar dominio del proyecto, explicando la lógica del programa, ejecutándolo correctamente y realizando modificaciones solicitadas por el docente.

Si durante la evaluación se evidencia que un estudiante:

- No comprende el código desarrollado,
- No puede explicar el análisis o las decisiones implementadas,
- No puede realizar modificaciones básicas solicitadas,
- Demuestra dependencia total de un compañero o de herramientas externas,

Esto será considerado incumplimiento de los objetivos formativos del curso y podrá calificarse como plagio o uso indebido de recursos, aplicándose una penalización individual de hasta el 100% de la nota del proyecto, según la gravedad del caso.

Criterio	Descripción	Valor (%)	Excelente (100%)	Bueno (75%)	Regular (50%)	Deficiente (25%)	No se evidencia (0%)
1. Manejo de estructuras de datos	El programa implementa correctamente todas las funciones solicitadas: ingreso de datos en Arbol B+, montículos máximo y mínimo.	50	Cumple todos los requerimientos sin fallos	Cumple $\geq 75\%$ de funcionalidades, con errores menores	Cumple $\geq 50\%$ con errores que afectan usabilidad	Cumple solo $\geq 25\%$ con muchas fallas y errores	Cumple menos del 30% de requerimientos y muestra errores al ejecutarse
2. Funcionalidades adicionales	Gestión correcta de la ubicación, existencia y carga de archivo con datos de entrada	15	Cumple todos los requerimientos sin fallos	Cumple $\geq 75\%$ de funcionalidades, con errores menores	Cumple $\geq 50\%$ con errores que afectan usabilidad	Cumple solo $\geq 25\%$ con muchas fallas y errores	Cumple menos del 30% de requerimientos y muestra errores al ejecutarse
3. Diseño y codificación	Claridad, modularidad, uso de funciones nativas para la gestión de archivos y estructuras.	10	Código modular, claro y documentado. Solo usa funciones nativas para gestionar archivos y sus estructuras	Código modular, en su mayoría bien documentado. Uso adecuado de funciones nativas para cadenas y propias para estructuras; con errores menores	Código poco modular o confuso. Dificultades en uso de funciones nativas para archivos y sus estructuras	Código desordenado y sin modularidad. Documentación insuficiente o ausente. Uso inadecuado de funciones nativas y propias	No presenta código funcional o incumple requisitos de diseño
4. Documentación técnica y manual de usuario	Manual técnico con diagramas de flujo, clases y explicación de implementación; manual de usuario con instrucciones claras.	5	Documentación completa, incluye los diagramas, la documentación es clara y ordenada	Documentación casi completa con mínimos vacíos, falta alguno de los diagramas	Documentación parcial, estructura poco clara, diagramas incompletos	Documentación deficiente, faltan los diagramas	Omite la documentación técnica y de usuario

5. Evidencia de competencias CG2 – Liderazgo solidario y transformador	Trabajo colaborativo, coordinación y aporte equitativo.	5	Colaboración efectiva, roles claros y equitativos	Colaboración adecuada con pequeños desequilibrios	Colaboración desigual, no colabora de forma constante	Se evidencia una colaboración mínima	No se evidencia colaboración alguna
6. Evidencia de competencias CG4 – Gestión del conocimiento técnico-científico	Integración y aplicación de conceptos de análisis léxico, estructuras de datos y generación de reportes.	10	Explica y aplica fundamentos teóricos de forma apropiada en todo el proyecto	Aplica la mayoría de conceptos	Aplicación parcial de teoría	Se evidencia una aplicación mínima de la teoría	Falta comprensión y aplicación de los conceptos teóricos solicitados
7. Evidencia de competencias CE9 – Gestión estratégica de TI	Desarrollo de un prototipo funcional que organiza información y genera reportes para toma de decisiones.	5	Prototipo sólido con integración completa de funcionalidades	Prototipo funcional con fallas menores	Prototipo limitado o incompleto, con errores que afectan significativamente el uso o funcionamiento	Prototipo incompleto, con fallas y errores frecuentes	No se evidencia gestión adecuada de TI
8. Presentación y dominio individual	Defensa oral presencial: cada estudiante debe explicar, ejecutar y realizar cambios al código.	-100	(-0) Explica con claridad y demuestra dominio total	(-10) Explica bien con leves dificultades	(-30) Explicación incompleta, dominio parcial	(-60) Explicación deficiente, poco dominio del proyecto realizado	(-90) No demuestra comprensión de lo implementado